Room Occupancy Estimation
Milestone 2

1st Manan Mishra
Student ID: 40321141
ma_mishr@live.concordia.ca

2nd Meet Maniar
Student ID: 40039203
m_aniar@live.concordia.ca

3rd Nazmus Sakib Khan
Student ID: 40309367
k_nazmu@live.concordia.ca

GitHub Repository: https://github.com/meetmaniar/INSE-6450

I. Model Selection (2.5 pts)

The Milestone 2 continues on prior analysis done in Milestone 1 . We retain **regression** as the primary task and evaluate predictions using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R-squared (R^2).

A. Candidate model families

TABLE 1
CANDIDATE MODEL FAMILIES AND THEIR TRADE-OFFS

Model family	Why suitable	Main limitations
Tree-Based Ensembles	Captures sharp non linear relations and also works without scaling inputs .	It is less intuitive than linear models.
Generalized Linear Models	Easily interpretable and offers low latency.	Struggles with non-linear relationships
Distance-Based Models	It learns complex local decision boundaries.	High Latency at prediction time.

B. Model choice and architectural details

1) Task Type Alignment: The task is a regression task as it evaluates Room Occupancy Count as a continuous count target. The model chosen for prediction is XGBRegressor.

2) 2. Alignment with Sensor Features:

- Axis-Aligned Decision Boundaries: Decision trees based models splits data using simple "if/else" rules. This matches physical sensor triggers, such as light turning on/off or motion sensors that detect movement ($S6_PIR, S7_PIR$).
- Non-Linear Feature Interactions: Trees automatically combine clues from multiple sensors across different branches (e.g., rising $S5_CO2_Slope$ paired with warmer Temperature without requiring manual math formulas or extra engineered features).
- Monotonic Scale Invariance: Tree based models only care about whether a value is higher or lower irrespective of units . Also these models can handle mixed sensor scales naturally without requiring an extra step to perform scaling.

3) 3. Speed, Memory, and Accuracy Trade-offs:

- Accuracy vs. Model Complexity: XGBRegressor is expected to deliver much higher predictive accuracy on tabular sensor data than linear regression as it will be able to cpature any non linear relationships also. the model should work without the training complexity or extreme GPU requirements of deep neural networks.
- Robustness: As XGBRegressor is a tree based model, tree depth limits and learning rate decay can be tuned to prevent overfitting to sensor noise and predict stable occupancy estimations.
- Inference Latency and Memory Footprint: XGBRegressor should be able to evaluates predictions in milliseconds and will have a model size of ($\sim$a few MBs) which makes it significantly faster and lighter on memory than vicinity based models like k-NN.

C. Architecture

1) Input representations preprocessing::

- Input Feature Vector (X): There are 7 input features consisting of spatial averages ($T_{\text{mean}}, L_{\text{mean}}, S_{\text{mean}}$), carbon dioxide metrics (`S5_CO2`, `S5_CO2_Slope`), and motion indicators (`S6_PIR`, `S7_PIR`).The means are calculated temperature light and sound as each measure is been reported by 4 sensors whose corelation among themselves it too high. Hence to not drop any sensor realted feature completely mean is been calculated to get an idea of the complete enclosed space.

- Dropped Columns: Explicit metadata columns (`Date`, `Time`, `Datetime`, and temporary `block_id`) were dropped before training to prevent temporal memorization. Model should predict the number of people in a room based on only sensor related information.
- Data Splitting: Grouped raw data into 120 contiguous time blocks (approximately $\sim$42 minutes each) and stratified blocks by peak occupancy into Train (60%), Validation (10%), and Test (30%) sets to avoid random shuffling. This is also done to make sure each split gets some representation of a each room occupancy.
- Feature Scaling: Features are fed as raw numerical values since XGBoost is scale-invariant.

2) Main architecture: The project uses XGBRegressor which builds an additive ensemble of shallow regression trees, each one sequentially fit to correct the residual error of the ensemble so far, scaled by a learning rate. The objective function lets each tree's output be interpreted as a log-rate for a non-negative count. Up to 500 trees are added, but training stops early once validation MAE fails to improve for 30 consecutive rounds, which is helps to determine the final model size.

3) Versioned configuration: Please refer to Table 2 for the full configuration settings.

TABLE 2
VERSIONED MODEL AND TRAINING CONFIGURATION

Parameter	**Configuration**
Random seed	42
Dataset split	Block Stratified
N_estimators	500 (Cap)
Learning rate	0.05
Max Depth	5
Objective	Poisson
Evaluation Metric	MAE
Early Stopping	30 Round Patience
Normalization	None
Tree method	Histogram
Data type / framework	XGBoost $\geq$ 2.0 Python 3.13

II. Training & Experimentation (3.0 pts)

A. Methodology, hyperparameters, and imbalance

A purely temporal 60/10/30 split (Table 3) was rejected: the dataset's large recording gaps placed 100% of the validation window inside a zero-occupancy period, with no representation of any other class. Hence, the chronologically-sorted data was partitioned into 120 contiguous time blocks ($\sim$85 rows / $\sim$42 min each), and whole blocks were split 60/10/30 via stratified sampling on each block's peak occupancy (Table 4). This keeps near-duplicate $\sim$30-second readings together while balancing occupancy classes across splits. Table 5 outlines the selected hyperparameters and tuning strategy.

TABLE 3
OCCUPANCY DISTRIBUTION ACROSS CHRONOLOGICAL PARTITIONS

Split	**Rows**	**0 people**	**1 person**	**2 people**	**3 people**
Train (60%)	6,077	73.56%	7.55%	10.78%	8.11%
Validation (10%)	1,013	100.00%	0.00%	0.00%	0.00%
Test (30%)	3,039	90.33%	0.00%	3.06%	6.61%

TABLE 4
TARGET CLASS DISTRIBUTION ACROSS STRATIFIED BLOCK-BASED PARTITIONS

Split	**Rows**	**0 people**	**1 person**	**2 people**	**3 people**
Train (60%)	6,077	82.64%	3.55%	7.19%	6.62%
Validation (10%)	1,015	66.70%	15.67%	7.98%	9.66%
Test (30%)	3,037	83.27%	2.77%	7.57%	6.39%

B. Results

Both models converge within $\sim$100 rounds: validation MAE flattens, then rises past the best iteration while training MAE keeps falling toward zero, confirming early stopping locks in the pre-overfit checkpoint in both cases. Table 7 summarizes the best early-stopping iterations for each model.

Class-imbalance mitigation. `Room_Occupancy_Count` is dominated by count 0 (81% of rows), and counts 2 and 3 feature data points don't distinguish by a considerable margin hence resulting into wrong predictions . To address this, rows with a true count of 2 or 3 were assigned 4$\times$ sample weight during training.

TABLE 5
HYPERPARAMETERS AND TUNING APPROACH

Setting	Value	Chosen via
`n_estimators`	500 (cap)	Manual default
`learning_rate`	0.05	Manual default
`max_depth`	5	Manual default
`objective`	`count:poisson`	Manual
Effective boosting rounds	Early-stopped	Automatic (30-round patience on validation MAE)

TABLE 6
HARDWARE AND SOFTWARE ENVIRONMENT

Component	Detail
Machine	Windows
GPU	None — CPU-only training
Tree method	`tree_method="hist"` (XGBoost default)
Python	3.13
Key packages	`xgboost>=2.0`, `scikit-learn>=1.3`, `pandas>=2.0`

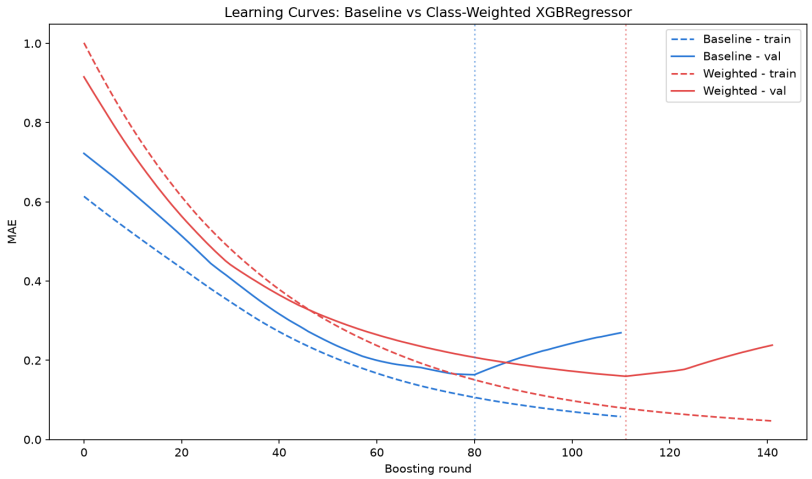


Fig. 1. Learning Curves.

TABLE 7
LEARNING CURVE SUMMARY (EARLY-STOPPING ITERATION)

Model	Best iteration	Max rounds
Baseline	80	500
Weighted	111	500

TABLE 8
BASELINE VS. CLASS-WEIGHTED MODEL COMPARISON

Model (Split)	MAE	RMSE	R^2
Baseline (Val)	0.164	0.353	0.872
Baseline (Test)	0.129	0.261	0.911
Weighted (Val)	0.160	0.332	0.888
Weighted (Test)	0.104	0.182	0.957

Effect: on the test set, every metric improves (MAE, RMSE, R^2), and the model now recovers count-3 predictions instead of collapsing them all into count 2.

C. Task Metrics (Regression)

The final model is the class-weighted `XGBRegressor`. The test set achieved a higher R^2 (0.957) and lower error metrics compared to validation ($R^2 = 0.888$, MAE $= 0.160$), which is attributed to the smaller 10% validation split ($\sim$12 contiguous blocks) exhibiting higher localized temporal variance than the broader 30% test split.

TABLE 9
FINAL MODEL REGRESSION METRICS

Split	MAE	RMSE	R^2
Validation	0.160	0.332	0.888
Test	0.104	0.182	0.957

III. Efficiency Metrics — Training & Inference (2.0 pts)

The scores are for final model which is class-weighted `XGBRegressor`.

1. Training Efficiency

A. Training Time: 1.544 s (3934.3 rows per second)

B. Peak Memory and VRAM Usage: Approximate RAM usage - 205.6 MB

C. Model Scale and Storage Size: Scale - 111 Trees , Size - 261.3 KB

D. Floating Point Operations (FLOPs): FLOPs a deep-learning metric for counting matrix-multiply operations not relevant for `XGBRegressor`

E. Energy Consumption and Cost Estimates: 0.013 Wh (0.001 g CO2eq, est. cost $0.00000@0.15$/kWh)

2. Inference Efficiency

A. Hardware Environment and Batch Configuration: Windows CPU: Intel64 Family 6 Model 140, GenuineIntel GPU: none (CPU-only)

B. Inference Latency (p50, p90, and Batch): Single-row (batch=1) latency scores are p50=3.169 ms, p90=3.602 ms (n=200). For Batch (batch=3038) latency: 5.50 ms total (0.0018 ms/row).

C. Throughput Performance: 552293.3 rows/s

D. Inference Memory Footprint: 225.8 MB

E. Model Optimization and Compression (ONNX/Quantization): No model compression has been implmented in the model but if it required we can convert the trained XGBRegressor model to ONNX format and apply post-training FP16 quantization to minimize model footprint and reduce CPU inference latency.

IV. Deployment Design & System-Level Trade-offs

A. System Deployment Architecture

1. Online vs. Offline Processing:

- Offline (Precomputed): Model training, hyperparameter tuning and cross-validation has to be done prior to deployment.
- Online (Real-Time): On-the-fly calculation of spatial mean features ($T_{\text{mean}}, L_{\text{mean}}, S_{\text{mean}}$), real-time `XGBRegressor` inference, prediction clipping $[0, 3]$ to predict a value between 0 to 3, and automated HVAC/lighting actuation triggers.

2. Deployment Topology (Edge / Local Gateway):

- Device Limits & Memory: The `XGBRegressor` model size ($\sim$few MBs) fits effortlessly into low-cost edge devices.
- Latency & Reliability: Enables fast local prediction independent of internet connectivity, preventing system downtime during network outages.
- Cost & Privacy: It will eliminate any API payload and cloud fees while protecting internal occupancy number by keeping raw sensor data inside the local network.

3. Workload Pattern (Streaming):

- Sensor Data collection Event-driven streaming processing operating on predetermined periodic cycles.
- Justification: Immediate occupancy estimation is required for HVAC optimization but the periodic cycles it should be calculated at should not be very short.

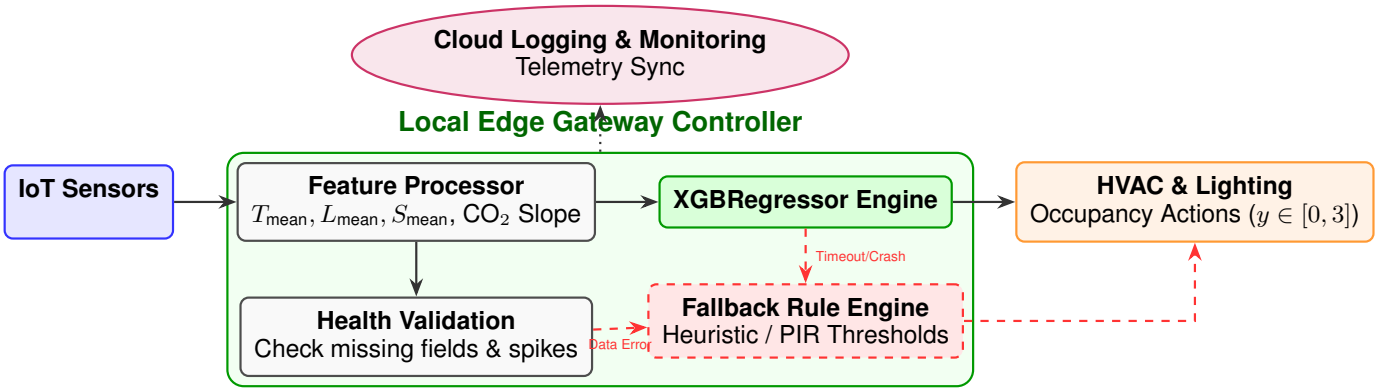


Fig. 2. High-Level Runtime System Architecture featuring Edge Inference, Real-Time Actuation, and Fallback/Rollback Mechanisms.

B. High-level system architecture and failure/rollback plan

As illustrated in Fig 2, the system performs real-time occupancy estimation through a local edge gateway. If sensor records spikes or have missing data, or if the model takes a long time to infer, the system automatically switches to the fallback rule engine using predefined PIR thresholds and data-error heuristics.

C. Latency and Accuracy Trade-Offs

1. Target Service Level Objectives (SLOs)

A. Latency Constraints (p95 Latency): Target p95 inference latency below 2 second on the local edge gateway to ensure timely HVAC and lighting adjustments.

B. Predictive Quality Baselines (Minimum Accuracy / Error Thresholds): Maintain MAE ≤ 0.20 and $R^2 \geq 0.85$, using the achieved test performance (MAE $= 0.104$, $R^2 = 0.957$) as a reference.

2. Shadow Deployment and A/B Testing Strategy

A. Shadow Mode Architecture: Run XGBRegressor alongside the existing rule-based system without triggering HVAC or lighting actions until model reliability is verified.

B. Traffic Splitting and Canary Rollout Plan: Gradually enable model-based occupancy control on a limited portion of operations while monitoring errors and reverting to fallback rules if necessary.

3. Real-Time Telemetry and System Monitoring

A. Operational Error Spikes and Failure Alerting: Monitor sensor anomalies, inference failures, and prediction errors through cloud logging, triggering alerts and fallback rules when necessary.

B. Data and Concept Drift Monitoring: Monitor changes in sensor distributions and prediction accuracy over time to identify potential drift and determine when retraining is required.